

Puzzle Schema v0.1 (Design Note)

Puzzle Metadata

- id (unique)
- name
- description
- tags (exploration, logic, combat, story, etc.)
- prerequisites (flags, items, other puzzles)
- mode: SEQUENCE | ANY_ORDER | BRANCHED
- repeatable (true/false)

Steps (Ordered List)

Each step has:

- type (canonical action type)
- params (object, npc, item, room, flag, etc.)
- optional (true/false)
- timeout (seconds, optional)
- hint_key (optional)
- on_complete (flags/events)

Canonical Step Types

- FIND_ITEM(item_id)
- CRAFT_ITEM(item_id)
- USE_ON(item, target)
- INTERACT(target, verb)
- GO_TO(room_id)
- TALK_TO(npc_id)
- ASK(npc_id, topic)
- GIVE(item_id, npc_id)
- DEFEAT_NPC(npc_id)
- TRIGGER(flag)
- SET_FLAG(flag=true/false)
- TIME_WINDOW(start, end)
- WAIT(duration)
- READ(object_id)
- SCAN(object_id)
- COMBINE(item_a, item_b, result)

Rewards

- items
- flags
- xp / skill unlocks
- access to rooms
- spawning NPCs or objects

Fail States

- conditions that fail puzzle
- consequences (flags, damage, lockouts)

Hints

- tiered hints
- unlocked by time or player request

Runtime Model

- state: LOCKED → ACTIVE → COMPLETED or FAILED
- current_index for SEQUENCE
- completed_steps[] for ANY_ORDER/BRANCHED
- progress stored in save data
- events: puzzle_started, step_completed, puzzle_completed, puzzle_failed

Engine Hooks

- Evaluator: listens for world events and attempts to advance puzzle
- Registry: maps step types to checking functions
- Adapter: puzzle completion triggers rewards, flags, spawns
- Hint system: reveal hints based on conditions

Notes

This is the minimal foundation. More complex branching, conditions, and scripting can be added later.